RETAIL SALES ANALYTICS | SQL PROJECT

SQL SERVER

EXPLORATORY DATA ANALYSIS (EDA) PROJECT

Made By Ardhendu

02 WHAT IS EXPLORATORY DATA ANALYSIS (EDA) ?

Exploratory Data Analysis (EDA) is the initial step in data analysis where data is explored, cleaned, and summarized to understand its structure, quality, and key patterns.

1
```sql
SELECT * FROM dim_customers
SELECT * FROM dim_products
SELECT * FROM fact_sales
```

2
```sql
--Explore All Objects in the Database
SELECT *
FROM INFORMATION_SCHEMA.TABLES
```

3
```sql
-- Checking data type of each column
SELECT
TABLE_NAME,
COLUMN_NAME,
DATA_TYPE
FROM INFORMATION_SCHEMA.COLUMNS;
```

Results | Messages

	TABLE_NAME	COLUMN_NAME	DATA_TYPE
1	dim_customers	customer_key	smallint
2	dim_customers	customer_id	smallint
3	dim_customers	customer_number	nvarchar
4	dim_customers	first_name	nvarchar
5	dim_customers	last_name	nvarchar
6	dim_customers	country	nvarchar
7	dim_customers	marital_status	nvarchar
8	dim_customers	gender	nvarchar
9	dim_customers	birthdate	date

4
```sql
-- Explore how many rows are there in each table?
SELECT 'dim_customers' AS TBL_NAME,COUNT(*) NUM_ROWS
FROM dim_customers
UNION ALL
SELECT 'dim_products' AS TBL_NAME,COUNT(*) NUM_ROWS
FROM dim_products
UNION ALL
SELECT 'fact_sales' AS TBL_NAME,COUNT(*) NUM_ROWS
FROM fact_sales
```

TBL_NAME	NUM_ROWS
dim_customers	18484
dim_products	295
fact_sales	60398

5
```sql
--Checking NULLS
SELECT
    'dim_customers' AS TBL_NAME,
    SUM(CASE WHEN customer_key IS NULL THEN 1 ELSE 0 END) AS customer_key,
    SUM(CASE WHEN customer_id IS NULL THEN 1 ELSE 0 END) AS customer_id,
    SUM(CASE WHEN customer_number IS NULL THEN 1 ELSE 0 END) AS customer_number,
    SUM(CASE WHEN first_name IS NULL THEN 1 ELSE 0 END) AS first_name,
    SUM(CASE WHEN last_name IS NULL THEN 1 ELSE 0 END) AS last_name,
    SUM(CASE WHEN country IS NULL THEN 1 ELSE 0 END) AS country,
    SUM(CASE WHEN marital_status IS NULL THEN 1 ELSE 0 END) AS marital_status,
    SUM(CASE WHEN gender IS NULL THEN 1 ELSE 0 END) AS gender,
    SUM(CASE WHEN birthdate IS NULL THEN 1 ELSE 0 END) AS birthdate,
    SUM(CASE WHEN create_date IS NULL THEN 1 ELSE 0 END) AS create_date
FROM dim_customers
```

6
```sql
-----CHECKING BLANKS
SELECT
    'dim_customers' AS TBL_NAME,
    SUM(CASE WHEN customer_key ='' THEN 1 ELSE 0 END) AS customer_key,
    SUM(CASE WHEN customer_id ='' THEN 1 ELSE 0 END) AS customer_id,
    SUM(CASE WHEN customer_number ='' THEN 1 ELSE 0 END) AS customer_number,
    SUM(CASE WHEN first_name ='' THEN 1 ELSE 0 END) AS first_name,
    SUM(CASE WHEN last_name ='' THEN 1 ELSE 0 END) AS last_name,
    SUM(CASE WHEN country ='' THEN 1 ELSE 0 END) AS country,
    SUM(CASE WHEN marital_status ='' THEN 1 ELSE 0 END) AS marital_status,
    SUM(CASE WHEN gender ='' THEN 1 ELSE 0 END) AS gender,
    SUM(CASE WHEN birthdate ='' THEN 1 ELSE 0 END) AS birthdate,
    SUM(CASE WHEN create_date ='' THEN 1 ELSE 0 END) AS create_date
FROM dim_customers
```

Results | Messages

	TBL_NAME	customer_key	customer_id	customer_number	first_name	last_name	country	marital_status	gender	birthdate
1	dim_customers	0	0	0	0	0	0	0	0	17

03 DIMENSION EXPLORATION & DATE EXPLORATION

Dimension Exploration is the process of analyzing categorical data (like country, category, gender) to understand distribution and unique values.
Understanding "who" and "what" in the data (not numbers, but categories)

```
--------------------DIMENSION EXPLORATION


-- Retrieve a list of unique countries from which customers originate
SELECT DISTINCT
    country
FROM dim_customers
ORDER BY country;

--Retrieve a list of unique Category from PRODUCT TBL
SELECT DISTINCT category
FROM dim_products

---Retrieve a list of unique SUB-Category from PRODUCT TBL
SELECT DISTINCT subcategory
FROM dim_products

-----Retrieve a list of unique product_name from PRODUCT TBL
SELECT DISTINCT product_name
FROM dim_products

-- Retrieve a list of unique categories, subcategories, and products
SELECT DISTINCT
    category,
    subcategory,
    product_name
FROM dim_products
ORDER BY category, subcategory, product_name;
```

Insight:
Customers are spread across multiple countries, indicating a geographically diverse customer base.
Helps identify potential target markets

Insight:
The dataset follows a hierarchical product structure: Category → Subcategory → Product.
Why this is helpful:
Analyze top-performing categories &
Drill down into subcategory/product performance

Date Exploration is the process of analyzing date-related data to understand time range, trends, and patterns.
Understanding "when" things are happening in the data

```
-----------------------------DATE EXPLORATION
-- Determine the first and last order date and the total duration in months
SELECT MIN(order_date) AS MIN_ORD_DATE, MAX(order_date) AS MAX_ORD_DATE,
DATEDIFF(MONTH,MIN(order_date), MAX(order_date)) AS ORDER_RANGE_YRS
FROM fact_sales

-- Find the youngest and oldest customer based on birthdate
SELECT
    MIN(birthdate) AS oldest_birthdate,
    DATEDIFF(YEAR, MIN(birthdate), GETDATE()) AS oldest_age,
    MAX(birthdate) AS youngest_birthdate,
    DATEDIFF(YEAR, MAX(birthdate), GETDATE()) AS youngest_age
FROM dim_customers;
```

Results | Messages

	oldest_birthdate	oldest_age	youngest_birthdate	youngest_age
1	1916-02-10	110	1986-06-25	40

Customer ages are analyzed based on birthdate; outliers are ignored for this practice.

Results | Messages

	MIN_ORD_DATE	MAX_ORD_DATE	ORDER_RANGE_YRS
1	2010-12-29	2014-01-28	37

What the query shows:
MIN(order_date) = 2010-12-29
MAX(order_date) = 2014-01-28
Total duration is approximately 37 months (~3 year

Insight:
The dataset spans ~3 years of sales data.

Why this is helpful:
- Trend analysis over time
- Seasonality detection
- Year-over-year comparisons

04 KPIS (KEY PERFORMANCE INDICATORS) & MAGNITUDE ANALYSIS

KPIs (Key Performance Indicators) are measurable values used to evaluate the performance of a business.
Numbers that show how well the business is doing

```sql
---- Generate a Report that shows all key metrics of the business
SELECT 'Total Sales' AS measure_name, SUM(sales_amount) AS measure_value FROM fact_sales
UNION ALL
SELECT 'Total Quantity', SUM(quantity) FROM fact_sales
UNION ALL
SELECT 'Average Price', AVG(price) FROM fact_sales
UNION ALL
SELECT 'Total Orders', COUNT(DISTINCT order_number) FROM fact_sales
UNION ALL
SELECT 'Total Products', COUNT(DISTINCT product_name) FROM dim_products
UNION ALL
SELECT 'Total Customers', COUNT(customer_key) FROM fact_sales
```

Results | Messages

	measure_name	measure_value
1	Total Sales	29356250
2	Total Quantity	60423
3	Average Price	486
4	Total Orders	27659
5	Total Products	295
6	Total Customers	60398

The business generated 29.36M in total sales from 27,659 orders, with 60,423 units sold and 60,398 customers. The average price per unit is 486, and there are 295 products in the catalog.

```sql
--Which gender made higher sales?
SELECT gender,SUM(sales_amount) AS total_sales
FROM fact_sales AS F
LEFT JOIN dim_customers AS C
ON F.customer_key =C.customer_key
GROUP BY gender
ORDER BY total_sales DESC
```

Results | Messages

	gender	total_sales
1	Female	14804168
2	Male	14522393

Magnitude Analysis is the process of comparing values across categories to understand which contributes more or less.

```sql
-- Find total customers by countries
SELECT  country,COUNT(customer_id) AS total_customers
FROM dim_customers
GROUP BY country
ORDER BY total_customers DESC;
```

country	total_customers
United States	7482
Australia	3591
United Kingdom	1913
France	1810
Germany	1780
Canada	1671

```sql
-- Find total customers by gender
SELECT  gender,COUNT(customer_id) AS total_customers
FROM dim_customers
GROUP BY gender
ORDER BY total_customers DESC;
```

gender	total_customers
Male	9341
Female	9128

```sql
-- Find total products by category
SELECT category,COUNT(product_id) AS total_products
FROM dim_products
GROUP BY category
ORDER BY  total_products DESC
```

Results | Messages

	category	total_products
1	Components	127
2	Bikes	97
3	Clothing	35
4	Accessories	29

```sql
-- What is the average costs in each category?
SELECT category,AVG(cost) AS  avgerage_cost
FROM dim_products
WHERE category IS NOT NULL
GROUP BY category
ORDER BY avgerage_cost DESC
```

	category	avgerage_cost
1	Bikes	949
2	Components	264
3	Clothing	24
4	Accessories	13

- The United States has the highest number of customers with 7,482, followed by Australia (3,591) and the United Kingdom (1,913).
- Male customers slightly outnumber female customers, with 9,341 males compared to 9,128 females.
- Components is the largest product category with 127 products, while Accessories has the fewest with 3 products.
- In terms of average cost, Bikes have the highest average cost at 949, much higher than the other categories.
- Sales are higher for females than males, with 14,840,168 in sales from female customers versus 14,522,393 from male customers.

05 RANKING ANALYSIS & CHANGE OVER TIME ANALYSIS

Ranking Analysis is used to identify top or bottom performers by ordering data based on a metric like sales or revenue.
"Who is best and who is worst"

```sql
--What are the top 5 revenue genrated products?
SELECT TOP 5 P.product_name,SUM(F.sales_amount) AS total_revenue
FROM fact_sales AS F
LEFT JOIN dim_products AS P
ON F.product_key = P.product_key
GROUP BY P.product_name
ORDER BY total_revenue DESC
```

	product_name	total_revenue
1	Mountain-200 Black- 46	1373454
2	Mountain-200 Black- 42	1363128
3	Mountain-200 Silver- 38	1339394
4	Mountain-200 Silver- 46	1301029
5	Mountain-200 Black- 38	1294854

```sql
-- What are the 5 worst-performing products in terms of sales?
SELECT *
FROM
    (SELECT P.product_name,SUM(F.sales_amount) AS total_revenue,
    DENSE_RANK() OVER(ORDER BY SUM(F.sales_amount)  ASC)  AS Rnk
    FROM fact_sales AS F
    LEFT JOIN dim_products AS P
    ON F.product_key = P.product_key
    GROUP BY P.product_name
    ) AS t
WHERE Rnk<=5
```

	product_name	total_revenue	Rnk
1	Racing Socks- L	2430	1
2	Racing Socks- M	2682	2
3	Patch Kit/8 Patches	6382	3
4	Bike Wash - Dissolver	7272	4
5	Touring Tire Tube	7440	5

```sql
-- Find the top 10 customers who have generated the highest revenue
SELECT *
FROM(
SELECT C.customer_id,CONCAT(C.first_name,' ',C.last_name) AS customer
,SUM(F.sales_amount) AS total_revenue,
ROW_NUMBER() OVER(ORDER BY SUM(F.sales_amount) DESC) AS row_num
FROM fact_sales AS F
LEFT JOIN dim_customers AS C
ON F.customer_key =C.customer_key
GROUP BY C.customer_id,CONCAT(C.first_name,' ',C.last_name)
) AS T
WHERE row_num<=10
```

customer_id	customer_name	total_revenue	row_num
12132	Kaitlyn Henderson	13294	1
12301	Nichole Nara	13294	2
12308	Margaret He	13268	3
12131	Randall Dominguez	13265	4
12300	Adriana Gonzalez	13242	5
12321	Rosa Hu	13215	6
12124	Brandi Gill	13195	7
12307	Brad She	13172	8
12296	Francisco Sara	13184	9
11433	Maurice Shan	12914	10

```sql
 -- The 3 customers with the fewest orders placed
SELECT TOP 3 C.customer_key, CONCAT(C.first_name, ' ',c.last_name) AS customer_name
,COUNT(DISTINCT order_number) AS total_orders
FROM fact_sales AS S
LEFT JOIN dim_customers AS C
ON C.customer_key =S.customer_key
GROUP BY C.customer_key,C.first_name,C.last_name
ORDER BY total_orders
```

customer_key	customer_name	total_orders
16	Chloe Young	1
17	Wyatt Hill	1
21	Jordan King	1

Change Over Time Analysis examines how data changes across different time periods.
"How things grow or decline over time"

```sql
==========================================================================
Change Over Time Analysis
==========================================================================
*/

-- Analyse sales performance over time
SELECT FORMAT(order_date,'yyyy-MM') AS Order_date,SUM(sales_amount) AS total_s
COUNT(DISTINCT customer_key) AS total_customers,
SUM(quantity) AS total_qty
FROM fact_sales AS S
WHERE order_date IS NOT NULL
GROUP BY FORMAT(order_date,'yyyy-MM')
ORDER BY Order_date ASC
```

Year_Month	total_sales
2010-12	43419
2011-01	469795
2011-02	488307
2011-03	405165
2011-04	502042
2011-05	561647
2011-06	737793
2011-07	598710
2011-08	614516
2011-09	603047
2011-10	708164
2011-11	662507
2011-12	689395
2012-01	495363
2012-02	506992
2012-03	373478
2012-04	400324
2012-05	358899
2012-06	555142
2012-07	444533
2012-08	523887
2012-09	488149
2012-10	536125
2012-11	537818
2012-12	624454
2013-01	857756
2013-02	771218
2013-03	1049732
2013-04	1045880
2013-05	1294456
2013-06	1642948
2013-07	1371595
2013-08	1545810
2013-09	1447324
2013-10	1673261
2013-11	1780888
2013-12	1874128
2014-01	45842

```sql
--Analyse monthly trends of total sales
SELECT
    FORMAT(order_date,'yyyy-MM') AS Year_Month,
    SUM(sales_amount) AS total_sales
FROM fact_sales
WHERE order_date IS NOT NULL
GROUP BY FORMAT(order_date,'yyyy-MM')
ORDER BY Year_Month;
```

```sql
--Analyse Yearly trends of total sales
SELECT YEAR(order_date) AS Years,SUM(sales_amount) AS total_sales
FROM fact_sales AS S
WHERE order_date IS NOT NULL
GROUP BY YEAR(order_date)
ORDER BY Years ASC
```

Years	total_sales
2010	43419
2011	7575088
2012	5842231
2013	16344878
2014	45842

Insight:

Ranking analysis shows that a few products drive most revenue, with Mountain-200 Black 46 leading sales, while lower-performing items such as Racing Socks-L, Racing Socks-M, Patch Kit/8 Patches, Bike Wash - Dissolver, and Touring Tire Tube generate the least revenue. Chloe Young, Wyatt Hill, and Jordan King are the least active customers, each placing only 1 order, which shows very low repeat purchase behavior. Change over time analysis shows strong growth from 2010 to 2013, with yearly sales peaking in 2013 at 16.34M. Monthly sales also rise steadily through the year, with the highest monthly sales recorded in December 2013 at 1,874,128, indicating a clear seasonal upward trend.

06 ADVANCE DATA ANALYTICS

These analyses help understand contribution, trends, growth, and customer behavior in data.

Cumulative Analysis calculates the running total of a metric over time.
"How values keep adding over time"

```sql
-- Calculate the total sales per year
-- and the running total sales over time
SELECT YEAR(order_date) AS Years,SUM(sales_amount) AS total_sales,
SUM(SUM(sales_amount)) OVER(ORDER BY YEAR(order_date)) AS running_total
FROM fact_sales AS S
WHERE order_date IS NOT NULL
GROUP BY YEAR(order_date)
```

Results | Messages

Years	total_sales	running_total
2010	43419	43419
2011	7075088	7118507
2012	5842231	12960738
2013	16344878	29305616
2014	45642	29351258

Data Segmentation divides data into groups based on conditions to better understand behavior.
"Grouping data into meaningful categories"

```sql
/*Segment products into cost ranges and
count how many products fall into each segment*/
WITH product_segments AS (
SELECT product_key,product_name,cost,
CASE WHEN cost<100 THEN 'Below 100'
     WHEN cost BETWEEN 100 AND 500 THEN '100-500'
     WHEN cost BETWEEN 500 AND 1000 THEN '500-1000'
     ELSE  'Above 1000'
END cost_range
FROM dim_products )

SELECT cost_range,COUNT(product_key) AS total_products
FROM product_segments
GROUP BY cost_range
ORDER BY total_products DESC
```

cost_range	total_products
Below 100	110
100-500	101
500-1000	45
Above 1000	39

Advanced analysis shows that sales accumulate steadily over time, with the running total increasing each year. Cumulative analysis highlights how total sales build up from year to year, while part-to-whole analysis shows that Bikes contribute the vast majority of total sales, making up 96.46% of revenue, followed by Accessories at 2.39% and Clothing at 1.16%. Data segmentation reveals that most products fall in the lower cost ranges, with the largest number of products priced below 100 and between 100–500. Performance analysis shows how product sales change over time, helping identify whether products are improving, declining, or remaining stable. Together, these analyses provide a clearer view of business growth, category contribution, and product performance trends.

Part-to-Whole Analysis shows how each category contributes to the total.
"Out of total, how much each part contributes"

```sql
-- Which categories contribute the most to overall sales?
SELECT *,
CONCAT(
        FORMAT(
        ROUND(
        CAST(total_sales AS Float)*100/SUM(total_sales) OVER() ,2)
        ,'N2'),'%') AS contribution
FROM (
SELECT P.category , SUM(S.sales_amount) AS total_sales
FROM fact_sales S
INNER JOIN dim_products AS P
ON S.product_key = P.product_key
GROUP BY P.category
) AS t
```

category	total_sales	contribution
Accessories	700262	2.39%
Bikes	28316272	96.46%
Clothing	339716	1.16%

Performance Analysis compares data across time periods like Year-over-Year (YoY) or Month-over-Month (MoM) to measure growth or decline.
"Is performance improving or declining?"

```sql
/* Analyze the yearly performance of products by comparing their sales
to both the average sales performance of the product and the previous year's sales */

WITH yearly_sales AS (
    SELECT YEAR(S.order_date) AS Year,P.product_name,SUM(sales_amount) AS current_sales
    FROM fact_sales AS S
    LEFT JOIN dim_products AS P
    ON S.product_key = P.product_key
    WHERE order_date IS NOT NULL
    GROUP BY YEAR(S.order_date),P.product_name
)
SELECT
YEAR,
product_name,
current_sales,
AVG(current_sales) OVER(PARTITION BY product_name ORDER BY YEAR ) avg_sales,
current_sales - AVG(current_sales) OVER(PARTITION BY product_name ) as diff_avg,
CASE WHEN current_sales - AVG(current_sales) OVER(PARTITION BY product_name )> 0 THEN 'Above Avg'
     WHEN current_sales - AVG(current_sales) OVER(PARTITION BY product_name )< 0 THEN 'Below Avg'
     ELSE 'Avg'
END AS avg_change,
LAG(current_sales) OVER(PARTITION BY product_name ORDER BY YEAR) AS prev_yr_value,
current_sales- LAG(current_sales) OVER(PARTITION BY product_name ORDER BY YEAR) AS YOY,
CASE WHEN current_sales -LAG(current_sales) OVER(PARTITION BY product_name ORDER BY YEAR)> 0 THEN 'Increase'
     WHEN current_sales - LAG(current_sales) OVER(PARTITION BY product_name ORDER BY YEAR)< 0 THEN 'Decrease
     ELSE 'No Change'
END AS yoy_change
FROM yearly_sales
ORDER BY product_name, YEAR
```

07 FINAL REPORTS – CUSTOMER REPORT

The Customer Report consolidates key customer metrics and behaviors into a single view for analysis. A complete summary of each customer's activity and value

1

```sql
CREATE VIEW customer_report AS
WITH base_query AS (
SELECT
S.order_number,
S.product_key,
S.order_date,
S.sales_amount,
S.quantity,
C.customer_key,
C.customer_number,
CONCAT(C.first_name, ' ', C.last_name) AS customer_name,
DATEDIFF(year, C.birthdate, GETDATE()) age
FROM fact_sales AS S
LEFT JOIN dim_customers C
ON S.customer_key = C.customer_key
WHERE S.order_date IS NOT NULL)

,customer_aggregation AS(
SELECT
    customer_key,
    customer_number,
    customer_name,
    age,
    COUNT(DISTINCT order_number) AS total_orders,
    SUM(sales_amount) AS total_sales,
    SUM(quantity) AS total_quantity,
    COUNT(DISTINCT product_key) AS total_products,
    MAX(order_date) AS last_order_date,
    DATEDIFF(month, MIN(order_date), MAX(order_date)) AS lifespan
FROM base_query
GROUP BY
customer_key,
customer_number,
customer_name,
age
)   SELECT
        customer_key,
        customer_number,
        customer_name,
        age,
        CASE
            WHEN age < 20 THEN 'Under 20'
            WHEN age between 20 and 29 THEN '20-29'
            WHEN age between 30 and 39 THEN '30-39'
            WHEN age between 40 and 49 THEN '40-49'
            ELSE '50 and above'
        END AS age_group,
```

2

```sql
    CASE
        WHEN lifespan >= 12 AND total_sales<= 5000 THEN 'Regular'
        WHEN lifespan >= 12 AND total_sales > 5000 THEN 'VIP'
        ELSE 'New'
    END AS customer_segment,
    last_order_date,
    DATEDIFF(month, last_order_date, GETDATE()) AS recency
    total_orders,
    total_sales,
    total_quantity,
    total_products,
    lifespan,
-- Compute average order value (AVO)
    CASE WHEN total_sales = 0 THEN 0
         ELSE total_sales / total_orders
    END AS avg_order_value,
-- Compute average monthly spend
    CASE WHEN lifespan = 0 THEN total_sales
        ELSE total_sales / lifespan
    END AS avg_monthly_spend
FROM customer_aggregation
```

```sql
Customer Report
==========================================================================
Purpose:
    - This report consolidates key customer metrics and behaviors

Highlights:
    1. Gathers essential fields such as names, ages, and transaction details.
    2. Segments customers into categories (VIP, Regular, New) and age groups.
    3. Aggregates customer-level metrics:
       - total orders
       - total sales
       - total quantity purchased
       - total products
       - lifespan (in months)
    4. Calculates valuable KPIs:
        - recency (months since last order)
        - average order value
        - average monthly spend
```

The customer report created using views and CTEs provides a complete profile of customers, including age, segment, recency, total orders, sales, quantity, product count, lifespan, average order value, and monthly spending.This report helps identify high-value customers, purchasing behavior, and segment performance for better business decision-making.

	customer_key	customer_number	customer_name	age	age_group	customer_segment	last_order_date	recency	total_orders	total_sales	total_quantity	total_products	lifespan	avg_order_value	avg_monthly_spend
1	1	AW00011000	Jon Yang	55	50 and above	VIP	2013-06-03	154	3	8249	8	8	26	2749	294
2	2	AW00011001	Eugene Huang	50	50 and above	VIP	2013-12-10	147	3	6384	11	10	35	2128	182
3	3	AW00011002	Ruben Torres	55	50 and above	VIP	2013-02-23	157	3	8114	4	4	25	2704	324
4	4	AW00011003	Christy Zhu	53	50 and above	VIP	2013-05-10	154	3	8139	9	9	29	2713	280
5	5	AW00011004	Elizabeth John...	47	40-49	VIP	2013-05-01	154	3	8196	6	6	28	2732	292
6	6	AW00011005	Julio Ruiz	50	50 and above	VIP	2013-05-02	154	3	8121	6	6	29	2707	280
7	7	AW00011006	Janet Alvarez	50	50 and above	VIP	2013-05-14	154	3	8119	5	5	28	2706	289
8	8	AW00011007	Marco Mehta	57	50 and above	VIP	2013-03-19	156	3	8211	8	8	26	2737	315
9	9	AW00011008	Rob Verhoff	51	50 and above	VIP	2013-03-02	156	3	8106	7	7	26	2702	311
10	10	AW00011009	Shannon Carla...	57	50 and above	VIP	2013-05-09	154	3	8091	5	5	28	2697	288
11	11	AW00011010	Jacquelyn Sua...	57	50 and above	VIP	2013-05-23	154	3	8088	4	4	28	2696	288
12	12	AW00011011	Curtis Lu	57	50 and above	VIP	2013-03-19	156	3	8133	4	4	27	2711	301
13	13	AW00011012	Lauren Walker	47	40-49	New	2013-10-15	148	2	81	5	5	7	40	11
14	14	AW00011013	Ian Jenkins	47	40-49	New	2014-01-21	146	2	114	5	5	9	57	12
15	15	AW00011014	Sydney Bennett	53	50 and above	New	2013-04-30	155	2	130	6	5	1	68	130

08 FINAL REPORTS – PRODUCT REPORT

Product Report is a final consolidated report that summarizes key product performance metrics and behavior.

```sql
CREATE VIEW report_products AS
WITH base_query AS (
SELECT
f.order_number,
f.order_date,
f.customer_key,
f.sales_amount,
f.quantity,
p.product_key,
P.product_name,
P.category,
P.subcategory,
P.cost
FROM fact_sales AS F
LEFT JOIN dim_products AS P
ON F.product_key = P.product_key
WHERE F.order_date IS NOT NULL
),
product_aggregations AS (
SELECT
    product_key,
    product_name,
    category,
    subcategory,
    cost,
    DATEDIFF(MONTH, MIN(order_date), MAX(order_date)) AS lifespan,
    MAX(order_date) AS last_sale_date,
    COUNT(DISTINCT order_number) AS total_orders,
    COUNT(DISTINCT customer_key) AS total_customers,
    SUM(sales_amount) AS total_sales,
    SUM(quantity) AS total_quantity,
    ROUND(AVG(CAST(sales_amount AS FLOAT) / NULLIF(quantity, 0)),1) AS avg
FROM base_query
GROUP BY
    product_key,
    product_name,
    category,
    subcategory,
    cost
)  SELECT
        product_key,
        product_name,
        category,
        subcategory,
        cost,
        last_sale_date,
        DATEDIFF(MONTH, last_sale_date, GETDATE()) AS recency_in_months,
        CASE
            WHEN total_sales > 50000 THEN 'High-Performer'
            WHEN total_sales >= 10000 THEN 'Mid-Range'
            ELSE 'Low-Performer'
        END AS product_segment,
        lifespan,
        total_orders,
        total_sales,
        total_quantity,
        total_customers,
        avg_selling_price,
        -- Average Order Revenue (AOR)
        CASE
            WHEN total_orders = 0 THEN 0
            ELSE total_sales / total_orders
        END AS avg_order_revenue,

        -- Average Monthly Revenue
        CASE
            WHEN lifespan = 0 THEN total_sales
            ELSE total_sales / lifespan
        END AS avg_monthly_revenue

FROM product_aggregations
```

```sql
Product Report
==============================================================================
Purpose:
    - This report consolidates key product metrics and behaviors.

Highlights:
    1. Gathers essential fields such as product name, category, subcategory, and cost.
    2. Segments products by revenue to identify High-Performers, Mid-Range, or Low-Performe
    3. Aggregates product-level metrics:
       - total orders
       - total sales
       - total quantity sold
       - total customers (unique)
       - lifespan (in months)
    4. Calculates valuable KPIs:
       - recency (months since last sale)
       - average order revenue (AOR)
       - average monthly revenue
```

The product report created using views and CTEs provides a complete profile of products, including product name, category, subcategory, cost, recency, lifespan, total orders, total sales, total quantity, total customers, average selling price, average order revenue, and average monthly revenue. This report helps identify high-performing products, product segment performance, and revenue contribution for better business decision-making.

	product_key	product_name	category	subcategory	cost	last_sale_date	recency_in_months	product_segment	lifespan	total_orders	total_sales	total_quantity	total_customers	avg_selling_price	avg_order_revenue	avg_month
1	48	Road-650 Red- 60	Bikes	Road Bikes	487	2012-12-23	159	Mid-Range	23	53	40071	53	53	756.1	756	1742
2	279	Short-Sleeve Cla	Clothing	Jerseys	42	2014-01-26	148	Mid-Range	12	409	22088	409	409	54	54	1840
3	132	Road-550-W Yell	Bikes	Road Bikes	713	2013-12-28	147	High-Performer	24	309	334440	309	303	1082.9	1082	13935
4	109	Mountain-500 Bl	Bikes	Mountain	295	2013-12-28	147	Mid-Range	11	56	30240	56	56	540	540	2749
5	3	Mountain-100 Bl	Bikes	Mountain	16	2011-12-27	171	High-Performer	11	49	165375	49	49	3375	3375	15034
6	148	Touring-3000 Bl	Bikes	Touring Bi	461	2013-12-17	147	Mid-Range	12	55	40810	55	55	742	742	3400
7	152	Touring-3000 Yel	Bikes	Touring Bi	461	2013-12-27	147	Mid-Range	12	47	34874	47	47	742	742	2908
8	295	Water Bottle - 30	Access	Bottles an	2	2014-01-20	148	Mid-Range	13	4244	21245	4240	4073	5	5	1634
9	235	Half-Finger Glov	Clothing	Gloves	6	2014-01-25	148	Mid-Range	13	487	11668	487	478	24	24	898
10	279	Short-Sleeve Cla	Clothing	Jerseys	42	2014-01-26	148	Mid-Range	13	374	20196	374	370	54	54	1553
11	128	Road-750 Black-	Bikes	Road Bikes	344	2013-12-28	147	High-Performer	12	380	194400	380	380	540	540	16200
12	272	Women's Mount	Clothing	Shorts	26	2014-01-29	148	Mid-Range	12	363	25410	363	360	70	70	2117
13	129	Road-750 Black-	Bikes	Road Bikes	344	2013-12-28	147	High-Performer	12	334	180360	334	334	540	540	15030
14	108	Mountain-500 Bl	Bikes	Mountain	295	2013-12-13	147	Mid-Range	12	48	25920	48	48	540	540	2160
15	6	Mountain-100 Bl	Bikes	Mountain	16	2011-12-28	171	High-Performer	12	57	192375	57	57	3375	3375	16031
16	292	Classic Vest- L	Clothing	Vests	24	2014-01-20	148	Mid-Range	12	195	12480	195	195	64	64	1040
17	43	Road-650 Red- 48	Bikes	Road Bikes	487	2012-12-27	159	High-Performer	23	88	66720	88	88	758.2	758	2900
18	49	Road-250 Red- 48	Bikes	Road Bikes	15	2012-12-24	159	High-Performer	12	162	365706	162	162	2443	2443	30480

09 SQL CONCEPTS & TECHNIQUES USED

Concepts and Methods Used

This project leverages key SQL concepts to perform structured and efficient data analysis:

- **CTEs (Common Table Expressions) and Views** were used to organize and simplify complex queries
- **Aggregate Functions** were applied to calculate key business metrics such as sales, orders, and quantity
- **Window Functions** were used for **ranking, trend analysis, and time-based comparisons**

These techniques enabled:

KPI Analysis
Ranking Analysis
Time-Based Trend Analysis
Customer Behavior Analysis
Product Performance Analysis

If you enjoyed this post, don't forget to like and comment.
I'll share the GitHub link for this project in the post description.